COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	09-16-2026	**DATE SUBMITTED:**	09-16-2026
NAME:	GUARIN, JAN MIKAELA C.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo2.h"

const int arrSize = 100;

void displayArr(int Arr[]);
int main()
{

    // seede the random number generator once for the whole program
    srand(time(0));

    // Array 1: Shell Sort

    int shellArr[arrSize];
    for (int i = 0; i < arrSize; i++)
    {
        shellArr[i] = rand() % 1000;
    }

    std::cout << "Array 1 - Unsorted:\n";
```

```
Array 1 - Unsorted:
993 799 709 304 96 792 139 985 56 50
197 977 331 888 402 367 843 948 226 49
465 996 292 751 62 965 142 374 591 211
320 47 822 768 178 79 303 147 168 663
688 914 610 566 141 51 545 174 362 23
543 396 59 416 63 468 160 589 869 781
61 684 974 841 889 575 243 656 701 578
314 29 976 27 166 601 536 771 136 494
611 788 382 870 351 412 930 726 767 769
990 205 415 909 432 874 458 85 504 125
```

Observations

I used srand(time(0)) to seed the random number generator, then filled an array of 100 integers using rand() % 1000. I did this for three separate arrays, one for each sorting algorithm. After running the program, I observed that each array had 100 random

	integers in no particular order, giving me unsorted data to test the sorting algorithms on.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
// Array 1: Shell Sort

int shellArr[arrSize];
for (int i = 0; i < arrSize; i++)
{
    shellArr[i] = rand() % 1000;
}

std::cout << "Array 1 - Unsorted:\n";
displayArr(shellArr);

shellSort(shellArr, arrSize);
std::cout << "\nArray 1 - Sorted using Shell Sort:\n";
displayArr(shellArr);
```

```
#ifndef SORT_ALGO_H
#define SORT_ALGO_H

// =================
// A.1. SHELL SORT
// =================
template <typename T>
void shellSort(T array[], int size)
{
    // for interval i <- size/2^n down to 1
    for (int interval = size / 2; interval > 0; interval = interval / 2)
    {

        // for each interval "i" in array
        for (int i = interval; i < size; i++)
        {

            // sort all the elements at interval "i"
            // (this is an insertion-sort style shift using the current interval)
            T temp = array[i];
            int j = i;

            while (j >= interval && array[j - interval] > temp)
            {
                array[j] = array[j - interval];
                j = j - interval;
            }
            array[j] = temp;
        }
    }
}
```

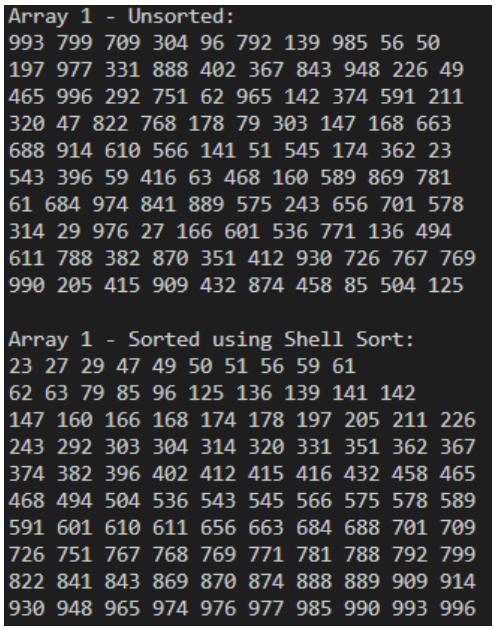
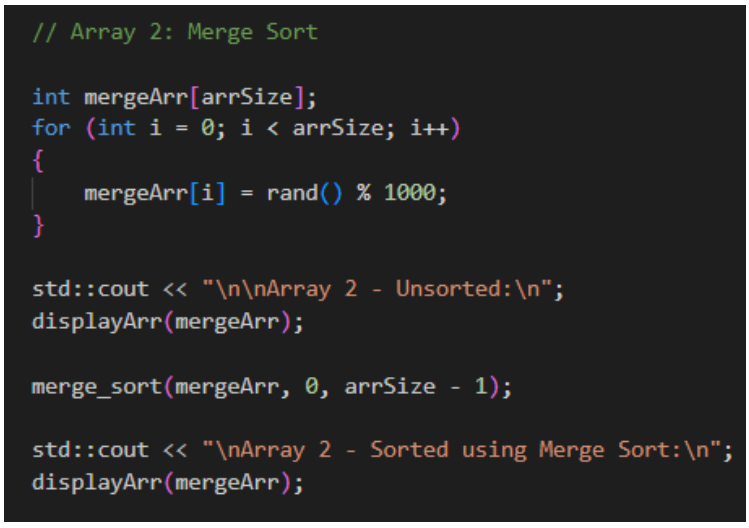
	
Observations	For Shell Sort, I used a shellSort() function that starts with a large gap (size/2) and keeps cutting it in half until it reaches 1. At each gap size, I compared elements that far apart and shifted them into place, similar to insertion sort. After running through all the gap sizes, I observed that the array was successfully sorted in ascending order.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```cpp
// ==================
// A.2. MERGE SORT
// ==================

// merge (Arr, left, middle, right) => merges the two sorted halves

template <typename T>
void merge(T arr[], int left, int middle, int right)
{
    int n1 = middle - left + 1;
    int n2 = right - middle;

    T leftArr[100];
    T rightArr[100];

    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2)
    {
        if (leftArr[i] <= rightArr[j])
        {
            arr[k] = leftArr[i];
            i++;
        }
        else
        {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    while (i < n1)
    {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
```

```
Array 2 - Unsorted:
682 956 921 243 327 893 311 88 379 504
926 425 566 580 996 824 406 215 88 443
314 330 762 70 502 89 444 80 681 569
642 636 445 642 726 450 503 351 111 405
344 731 667 973 19 71 723 976 24 618
886 756 888 736 752 633 514 518 692 381
61 172 648 711 927 91 464 8 614 490
638 455 69 724 736 668 417 876 210 111
741 900 606 714 698 90 305 748 75 676
551 839 224 445 522 165 33 712 727 727

Array 2 - Sorted using Merge Sort:
8 19 24 33 61 69 70 71 75 80
88 88 89 90 91 111 111 165 172 210
215 224 243 305 311 314 327 330 344 351
379 381 405 406 417 425 443 444 445 445
450 455 464 490 502 503 504 514 518 522
551 566 569 580 606 614 618 633 636 638
642 642 648 667 668 676 681 682 692 698
711 712 714 723 724 726 727 727 731 736
736 741 748 752 756 762 824 839 876 886
888 893 900 921 926 927 956 973 976 996
```

Observations	For Merge Sort, I used a merge_sort() function that recursively splits the array in half until each piece has only one element, then a merge() function that combines the sorted halves back together in order. After the recursion finished combining everything, I observed that the array was successfully sorted in ascending order.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```
    // Array 3: Quick Sort

    int quickArr[arrSize];
    for (int i = 0; i < arrSize; i++)
    {
        quickArr[i] = rand() % 1000;
    }

    std::cout << "\n\nArray 3 - Unsorted:\n";
    displayArr(quickArr);

    quicksort(quickArr, 0, arrSize - 1);

    std::cout << "\nArray 3 - Sorted using Quick Sort:\n";
    displayArr(quickArr);

    return 0;
}

void displayArr(int arr[])
{
    for (int i = 0; i < arrSize; i++)
    {
        std::cout << arr[i] << " ";
        if ((i % 10) == 9)
        {
            std::cout << '\n';
        }
    }
}
```

```
// =================
// A.3. QUICK SORT
// =================

// partition(A, low, high)
template <typename T>
int partition(T A[], int low, int high)
{
    T pivotValue = A[high]; // last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (A[j] < pivotValue)
        {
            i++;
            T temp = A[i];
            A[i] = A[j];
            A[j] = temp;
        }
    }

    T temp = A[i + 1];
    A[i + 1] = A[high];
    A[high] = temp;

    return i + 1;
}
```

```
// quicksort(A, low, high)
template <typename T>
void quicksort(T A[], int low, int high)
{
    // if(low < high)
    if (low < high)
    {
        // pivot = partition(A,low,high);
        int pivot = partition(A, low, high);

        // quicksort(A,low,pivot-1)
        quicksort(A, low, pivot - 1);

        // quicksort(A,pivot+1,high)
        quicksort(A, pivot + 1, high);
    }
}

#endif
```

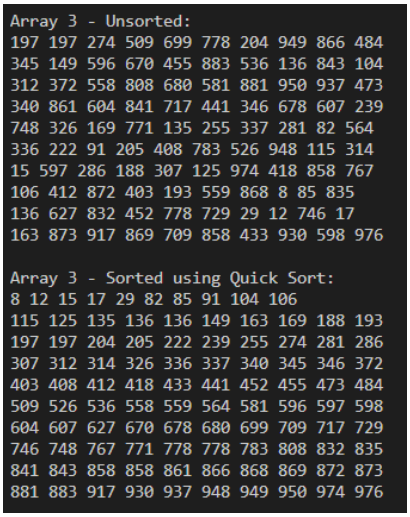
	
Observations	For Quick Sort, I used a quicksort() function with a partition() helper that picks the last element as the pivot and moves smaller elements to its left and larger ones to its right. I then recursively sorted the left and right sides. After the recursion completed, I observed that the array was successfully sorted in ascending order.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, we can. After partition() splits the array into a left sublist (smaller than the pivot) and a right sublist (larger than the pivot), those sublists are just regular arrays, so any sorting algorithm can be used on them instead of calling quicksort() again. For example, with the array {12, 8, 3, 15, 20} and pivot = 12, we'd get left sublist {8, 3} and right sublist {15, 20}. Instead of recursively calling quicksort() on {8, 3}, we could use shellSort() on it instead, and it would still correctly sort it into {3, 8}.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- I think Quick Sort would be the fastest for this array since it's a small list and Quick Sort doesn't need extra arrays like Merge Sort does, so it has less overhead. As for why Merge Sort and Quick Sort are both O(N log N), it's because they both work by splitting the array in half over and over. We can only cut an array in half so many times before we're down to single elements, and that number of times is log N. At each of those "levels," the algorithm still has to go through all N elements to merge them back together or partition them around a pivot. So we end up multiplying the log N levels by the N work per level, which gives us N log N overall.

CONCLUSION:
At first, sorting algorithms seemed pretty challenging and confusing to me, but this activity really helped me understand how Shell Sort, Merge Sort, and Quick Sort work and how they're different from each other. Shell Sort compares elements far apart using shrinking gaps, Merge Sort splits the array and merges the sorted halves back together, and Quick Sort uses a pivot to partition and sort the array.

For the procedure, generating random arrays and testing each algorithm separately, then checking the console output to confirm they sorted correctly, really helped me see how each algorithm works step by step instead of just reading about it.

For the supplementary activity, I learned that quicksort's sublists can actually be sorted using other algorithms, not just quicksort itself, and I got a better understanding of why Merge Sort and Quick Sort both share the O(N log N) time complexity because of how they divide the array.

I think I did well in this activity since all three algorithms worked correctly in the end. One area I could still improve on is understanding time complexity more deeply, especially the worst-case scenarios, and getting more practice tracing through the algorithms by hand instead of just relying on the code running.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.